Übung 03
CNN mit **Keras** in **Python**: Erkennung mit eigenen Bildern
INFI-IS
5xHWII

November 2, 2026

Abgabetermin: lt. Moodle
Übungsleiter: Albert Greinöcker



CCA – COMPETENCE CENTRE
HTL Anichstraße

Ziel der Übung:

- Kennenlernen von `Convolutional Neural Networks (CNN)` mit `Keras`
- Erstellen eines eigenen Bilddatensatzes
- Verstehen, warum die Qualität und Vielfalt der Trainingsdaten entscheidend ist
- Aufteilen der Daten in Trainings-, Validierungs- und Testdaten
- Untersuchen von *Overfitting* und *Generalisierung*
- Verbessern des Modells durch *Data Augmentation*
- Vergleich eines selbst aufgebauten CNN mit einem vortrainierten Modell (*Transfer Learning*)
- Anwenden des trainierten Modells auf neue Bilder bzw. Kamerabilder

1 Aufgabenstellung

In dieser Übung soll ein eigenes CNN zur Klassifikation von Bildern entwickelt werden. Im Gegensatz zu den bisherigen MNIST-Beispielen werden die Trainingsdaten diesmal selbst erstellt. Ziel ist nicht nur eine möglichst hohe Trefferquote, sondern vor allem herauszufinden, **wann ein Modell auch mit wirklich neuen Bildern funktioniert**.
Als Klassen sollen **mindestens drei unterschiedliche Gegenstände** verwendet werden. Geeignet sind Gegenstände, die einfach verfügbar sind und sich grundsätzlich voneinander unterscheiden, zum Beispiel:

- Trinkflasche, Taschenrechner und Schlüsselbund
- Maus, Smartphone und Stift
- Tasse, Kopfhörer und Fernbedienung

Andere Gegenstände können natürlich ebenfalls verwendet werden. Gesten wie Schere–Stein–Papier sind für diese erste Übung mit eigenen Bildern eher ungeeignet, da sich dabei zusätzlich Handhaltung, Person, Hautfarbe, Hintergrund, Abstand und Perspektive stark verändern können.

2 Vorbereitung

- `Keras` sollte bereits aus Übung 02 installiert sein.
- Für die Aufnahme von Bildern kann `OpenCV` verwendet werden. Installation: `pip install opencv-python`; Import in Python: `import cv2`.
- Die in den Beispielen angegebenen lokalen Verzeichnisse müssen an die eigene Umgebung angepasst werden.

3 Erstellen des eigenen Datensatzes

Erstellt zunächst einen eigenen Datensatz mit mindestens drei Klassen. Pro Klasse sollen ungefähr **50–100 Bilder** aufgenommen werden. Entscheidend ist dabei nicht, möglichst schnell viele fast identische Bilder zu erzeugen. Die Bilder sollen bewusst unterschiedlich sein.
Variiert daher unter anderem:

- Position des Gegenstandes im Bild
- Abstand zur Kamera und damit die Größe des Gegenstandes
- Blickwinkel und Rotation
- Hintergrund
- Beleuchtung

Für die Aufnahme kann das Programm `capture_images.py` aus den gemeinsamen Beispielen verwendet und angepasst werden.

Wichtiger Test

Ein Teil der späteren Testbilder soll **nicht während der eigentlichen Datensammlung entstehen**. Fotografiert die Gegenstände später noch einmal unter leicht anderen Bedingungen. Wenn möglich, soll zusätzlich ein anderer Gegenstand derselben Art verwendet werden, beispielsweise eine andere Computermaus oder eine andere Trinkflasche.
Damit soll überprüft werden, ob das CNN tatsächlich die *Klasse* gelernt hat oder nur den konkreten Gegenstand bzw. den Hintergrund aus den Trainingsbildern wiedererkennt.

4 Aufteilen und Einlesen der Daten

Teilt die Bilder in Trainings-, Validierungs- und Testdaten auf. Achtet darauf, dass Bilder derselben Aufnahmeserie nicht einfach zufällig auf alle drei Bereiche verteilt werden, da sich unmittelbar hintereinander aufgenommene Bilder häufig fast nicht unterscheiden.
Eine mögliche Verzeichnisstruktur ist:

```
data/
    train/
        klasse1/
        klasse2/
        klasse3/
    validation/
        klasse1/
        klasse2/
        klasse3/
    test/
```

```
        klasse1/
        klasse2/
        klasse3/
```

Keras kann eine solche Verzeichnisstruktur beispielsweise mit `image_dataset_from_directory()` direkt einlesen. Die Bilder sollen beim Laden auf eine einheitliche Größe gebracht werden, beispielsweise 128×128 Pixel.

5 Eigenes CNN erstellen

Erstellt nun ein eigenes CNN und trainiert es mit euren Trainingsdaten. Das Modell soll **nicht einfach aus einem fertigen Beispiel kopiert** werden. Überlegt selbst, wie viele Convolution- und Pooling-Schichten sinnvoll sind.
Eine mögliche grundsätzliche Struktur ist:

```
Input
  |
Conv2D
  |
MaxPooling
  |
Conv2D
  |
MaxPooling
  |
Conv2D
  |
GlobalAveragePooling oder Flatten
  |
Dense
  |
Ausgabe
```

Dokumentiert für euer Modell:

- die verwendete Architektur,
- die Anzahl der trainierbaren Parameter,
- die erreichte Accuracy auf Trainings- und Validierungsdaten,
- die Accuracy auf den Testdaten.

Stellt den Verlauf von *Loss* und *Accuracy* für Training und Validierung grafisch dar. Beurteilt anhand dieser Kurven, ob Anzeichen für **Overfitting** vorhanden sind.

6 Generalisierung testen

Nun kommt der wichtigste Teil der Übung: Testet das Modell mit Bildern, die sich deutlich von den Trainingsbildern unterscheiden.
Probiert beispielsweise:

- einen bisher nicht verwendeten Hintergrund,
- eine andere Beleuchtung,

- einen anderen Abstand zur Kamera,
- eine andere Lage des Gegenstandes,
- wenn möglich einen anderen Gegenstand derselben Klasse.

Dokumentiert einige Beispiele, bei denen die Vorhersage funktioniert, und einige, bei denen das Modell falsch liegt. Überlegt jeweils, **warum** das Modell Schwierigkeiten haben könnte.

Zusatzexperiment: Dataset Bias

Optional kann absichtlich ein schlechter Datensatz erzeugt werden. Fotografiert beispielsweise jede Klasse immer vor einem anderen Hintergrund. Wenn das Modell damit eine hohe Accuracy erreicht, testet anschließend einen Gegenstand vor dem "falschen" Hintergrund.
Überlegt: Hat das CNN tatsächlich den Gegenstand erkannt oder möglicherweise hauptsächlich den Hintergrund gelernt?

7 Data Augmentation

Verbessert nun das Training durch *Data Augmentation*. Dabei werden während des Trainings zufällige Veränderungen an den Bildern durchgeführt, zum Beispiel:

- Rotation
- Verschiebung
- Zoom
- horizontales Spiegeln, sofern dies für eure Gegenstände sinnvoll ist

Keras stellt dafür beispielsweise die Layer `RandomRotation`, `RandomTranslation` und `RandomZoom` zur Verfügung.

```

augmentation = keras.Sequential(
[
  layers.RandomRotation(0.05),  # +/- 18 degrees
  layers.RandomTranslation(0.1, 0.1),  # shift up to 10% in each direction
  layers.RandomZoom(0.1),
  ],
  name="augmentation",
)
```

Dieser Teil wird dann direkt ins Modell eingebaut:

```
model = keras.Sequential(
[
keras.Input(shape=input_shape),
  augmentation,
  ..
]
```

Trainiert das Modell erneut und vergleicht die Ergebnisse mit dem ersten Modell. Beantwortet dabei insbesondere folgende Fragen:

- Verändert sich die Accuracy auf den Trainingsdaten?

- Verändert sich die Accuracy auf den Validierungs- und Testdaten?
- Funktioniert das Modell bei euren neu aufgenommenen Bildern besser?
- Ist das Overfitting geringer geworden?

8 Vorhersage mit neuen Bildern bzw. Kamera

Speichert das trainierte Modell und erstellt anschließend ein separates Programm für die Vorhersage.
Das Programm soll:

- das gespeicherte Modell laden,
- ein neues Bild oder einen Kamera-Frame einlesen,
- das Bild genauso vorbereiten wie die Trainingsbilder,
- die Vorhersage des CNN berechnen,
- die erkannte Klasse und die zugehörige Wahrscheinlichkeit ausgeben.

Optional kann die Vorhersage direkt in einem Live-Kamerabild angezeigt werden. Achtet darauf, dass eine hohe ausgegebene Wahrscheinlichkeit nicht automatisch bedeutet, dass die Vorhersage korrekt ist.

9 Transfer Learning

Im letzten Teil soll untersucht werden, ob ein bereits auf einer großen Bildsammlung vortrainiertes CNN mit eurem kleinen Datensatz bessere Ergebnisse erzielt.
Verwendet dafür ein in Keras verfügbares vortrainiertes Modell, beispielsweise `MobileNetV2`, `EfficientNet` oder `ResNet`. Ein Beispiel für die Verwendung befindet sich unter: `ex_04_existing_cnns_resnet.py`.

Verwendet zunächst den vortrainierten Teil des Netzes als **Feature Extractor** und trainiert nur einen neuen Klassifikationskopf für eure eigenen Klassen.

Vergleicht anschließend das vortrainierte Modell mit eurem selbst entwickelten CNN:

- Welches Modell erreicht auf den Testdaten die bessere Accuracy?
- Welches Modell funktioniert bei wirklich neuen Fotos besser?
- Wie unterscheiden sich Trainingszeit und Anzahl der trainierten Parameter?
- Warum kann ein vortrainiertes CNN bereits mit relativ wenigen eigenen Bildern gute Ergebnisse erzielen?

Optional kann anschließend ein Teil des vortrainierten Netzes freigegeben und mit einer kleinen Lernrate weitertrainiert werden (*Fine-Tuning*).

10 Programme und Ergebnisse

Diese Programme und Ergebnisse sollten entstehen

- der Python-Code für Dateneinlesen, Training und Vorhersage,
- das selbst erstellte CNN,
- die Variante mit Data Augmentation,
- die Variante mit Transfer Learning,
- Accuracy- und Loss-Diagramme,
- einige aussagekräftige Testbilder mit korrekten und falschen Vorhersagen,
- eine kurze schriftliche Beantwortung der folgenden Fragen.

Bitte noch Folgendes überlegen

1. Warum reicht es nicht aus, nur die Accuracy auf den Trainingsdaten zu betrachten?
2. Woran kann man Overfitting erkennen?
3. Warum sind 100 fast identische Bilder schlechter als 100 möglichst unterschiedliche Bilder?
4. Was bewirkt Data Augmentation?
5. Was versteht man bei einem Bilderkennungsmodell unter Generalisierung?
6. Warum kann ein Modell eine hohe Test-Accuracy erreichen und trotzdem bei einem neuen Kamerabild schlecht funktionieren?
7. Was ist der wesentliche Unterschied zwischen eurem selbst trainierten CNN und Transfer Learning?
8. Welche Variante hat bei eurem Datensatz am besten funktioniert und welche Beobachtungen erklären das Ergebnis?

Weiterführende Überlegung

Euer Modell klassifiziert immer das gesamte Bild. Was müsste sich ändern, wenn in einem Kamerabild gleichzeitig mehrere Gegenstände vorkommen und zusätzlich erkannt werden soll, **wo** sich die einzelnen Gegenstände befinden? Informiert euch dazu über den Unterschied zwischen *Image Classification* und *Object Detection*.